

Informe del paper: *ZooKeeper: Wait-free coordination for Internet-scale systems*

Student:

López Arteaga, César Omar

Universidad Nacional de Ingeniería, Facultad de Ciencias

e-mail: clopeza@uni.pe

Curso:

Programación Concurrente y Distribuida - CC4P1A

Informe de lectura técnica

Junio 2026

Abstract

Este informe analiza el artículo *ZooKeeper: Wait-free coordination for Internet-scale systems*, escrito por Patrick Hunt, Mahadev Konar, Flavio P. Junqueira y Benjamin Reed. El trabajo presenta ZooKeeper, un servicio de coordinación para sistemas distribuidos a escala de Internet. Su idea central es no ofrecer primitivas bloqueantes rígidas en el servidor, sino un núcleo de coordinación de alto rendimiento basado en objetos *wait-free*, znodes jerárquicos, notificaciones mediante *watches*, orden FIFO por cliente y escrituras linealizables. Esta combinación permite construir, desde el cliente, primitivas como elección de líder, membresía de grupo, configuración dinámica, locks y barreras. El paper también muestra una implementación replicada basada en un protocolo de difusión atómica liderado, Zab, y evalúa su rendimiento bajo cargas dominadas por lecturas.

Keywords: ZooKeeper, coordinación distribuida, Zab, consenso, sistemas replicados, watches, znodes, linearizabilidad, wait-free.

Contents

1	Introducción	3
2	Ficha técnica del paper	3
3	Problema abordado	3
4	Idea central: núcleo de coordinación wait-free	4
5	Modelo de datos: znodes y árbol jerárquico	4
5.1	Tipos de znodes	4
6	API principal de ZooKeeper	5
7	Watches: notificaciones sin polling	5
8	Garantías de consistencia y orden	6

9 Implementación del servicio	6
9.1 Base de datos replicada	6
9.2 Zab y quorum	7
10 Recetas de coordinación implementables con ZooKeeper	7
10.1 Configuración dinámica	7
10.2 Membresía de grupo	7
10.3 Lock sin efecto manada	7
10.4 Barreras dobles	7
11 Aplicaciones descritas en el paper	8
12 Evaluación experimental	8
12.1 Throughput bajo cargas de lectura	8
12.2 Latencia y barreras	9
13 Comparación conceptual con Chubby	9
14 Análisis crítico	9
14.1 Fortalezas	9
14.2 Limitaciones	9
15 Conclusiones	10

1 Introducción

Las aplicaciones distribuidas de gran escala necesitan mecanismos de coordinación para resolver problemas como configuración dinámica, detección de fallos, membresía de grupo, elección de líder, sincronización y control de acceso a recursos compartidos. Tradicionalmente, estos problemas se han atacado con servicios especializados, por ejemplo colas, servicios de configuración o servicios de locks. El paper de ZooKeeper propone una estrategia distinta: construir un núcleo general de coordinación, simple y de alto rendimiento, sobre el cual los clientes implementan las primitivas que necesitan.

La contribución principal del artículo es mostrar que no es necesario colocar todas las primitivas complejas dentro del servidor. En lugar de eso, ZooKeeper expone una interfaz pequeña, parecida a un sistema de archivos jerárquico, con garantías de orden suficientes para que los clientes construyan abstracciones más fuertes. Esta decisión reduce la complejidad del servidor y evita que clientes lentos o defectuosos bloqueen el progreso global del sistema.

2 Ficha técnica del paper

Table 1: Datos principales del artículo analizado.

Título	<i>ZooKeeper: Wait-free coordination for Internet-scale systems</i>
Autores	Patrick Hunt, Mahadev Konar, Flavio P. Junqueira y Benjamin Reed
Afiliación	Yahoo! Grid y Yahoo! Research
Tema central	Coordinación de procesos en sistemas distribuidos a gran escala
Sistema propuesto	ZooKeeper, un servicio replicado de coordinación basado en znodes y watches
Idea clave	Delegar la construcción de primitivas de coordinación a los clientes, manteniendo en el servidor un núcleo simple, replicado y de alto rendimiento

3 Problema abordado

En un sistema distribuido, varios procesos ejecutan en máquinas distintas y deben tomar decisiones consistentes a pesar de fallos, retrasos de red y concurrencia. Los problemas recurrentes son:

- **Configuración dinámica:** los procesos deben leer y actualizar parámetros de operación sin detener el sistema.
- **Membresía de grupo:** cada proceso necesita saber qué nodos están activos.
- **Elección de líder:** varios procesos deben acordar cuál actuará como coordinador.
- **Sincronización:** algunos recursos requieren acceso exclusivo o barreras de coordinación.
- **Recuperación ante fallos:** el sistema debe continuar operativo mientras exista un quorum de servidores.

La dificultad está en que una primitiva bloqueante mal diseñada puede degradar todo el sistema. Por ejemplo, un lock mantenido por un cliente lento o caído puede impedir el avance de otros clientes. ZooKeeper evita esta dependencia directa: el servidor no implementa locks como primitiva básica, sino objetos simples sobre los que los clientes construyen locks, barreras o elección de líder.

4 Idea central: núcleo de coordinación wait-free

Definición 4.1 (Operación wait-free). *Una operación es wait-free si su terminación no depende de que otros clientes avancen. En el contexto de ZooKeeper, las operaciones básicas sobre znodes no esperan a que otros procesos liberen locks o respondan notificaciones.*

El diseño de ZooKeeper privilegia operaciones no bloqueantes en el servidor. Esto permite procesar muchas solicitudes en tubería, mantener baja latencia y evitar que un cliente defectuoso contamine el rendimiento de los demás. La responsabilidad de esperar, si una primitiva lo requiere, se implementa en el cliente usando *watches*; por ejemplo, un cliente que intenta adquirir un lock puede esperar la eliminación de un znode específico.

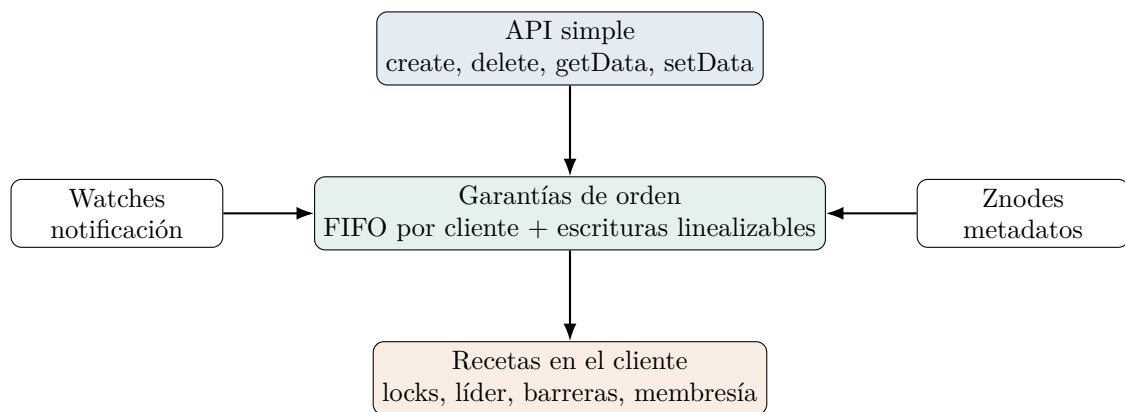


Figure 1: Idea conceptual de ZooKeeper: un servidor simple ofrece objetos y garantías mínimas; el cliente compone primitivas más complejas.

5 Modelo de datos: znodes y árbol jerárquico

El espacio de nombres de ZooKeeper se organiza como un árbol similar al de un sistema de archivos. Cada nodo se denomina *znode*. Un znode almacena datos y metadatos, como versiones y marcas temporales. A diferencia de un sistema de archivos general, los znodes están pensados para metadatos de coordinación, no para almacenamiento masivo de datos.

5.1 Tipos de znodes

Table 2: Tipos de znodes usados en ZooKeeper.

Tipo	Significado
Regular	Se crea y elimina explícitamente por el cliente. Es útil para configuración persistente o metadatos de larga duración.
Ephemeral	Está asociado a una sesión. Si la sesión termina por cierre normal o fallo, el znode se elimina automáticamente. Es fundamental para membresía de grupo y detección de procesos vivos.
Sequential	Al crearlo, ZooKeeper agrega un contador monótonamente creciente al nombre. Sirve para ordenar solicitudes concurrentes, por ejemplo en locks sin efecto manada.

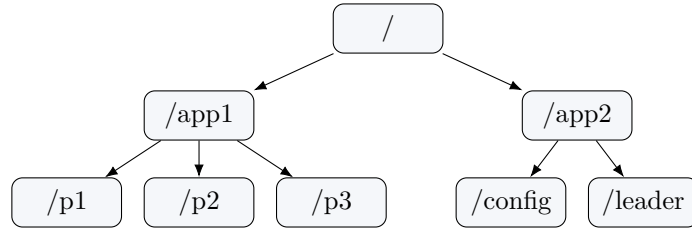


Figure 2: Ejemplo de espacio de nombres jerárquico con znodes. Los nodos `/app1/p_i` pueden representar procesos activos de una aplicación.

6 API principal de ZooKeeper

La API es pequeña y evita conceptos como `open()` o `close()` sobre archivos. Cada solicitud indica la ruta completa del znode. Las operaciones relevantes son:

Table 3: Operaciones principales de la API.

Operación	Descripción
<code>create(path,data,flags)</code>	Crea un znode en la ruta indicada. Los flags permiten crear nodos regulares, efímeros o secuenciales.
<code>delete(path,version)</code>	Elimina el znode si coincide la versión esperada.
<code>exists(path,watch)</code>	Verifica si existe un znode; opcionalmente registra un watch.
<code>getData(path,watch)</code>	Lee datos y metadatos del znode; opcionalmente registra un watch.
<code>setData(path,data,version)</code>	Escribe datos si la versión actual coincide con la esperada.
<code>getChildren(path,watch)</code>	Devuelve los hijos de un znode; opcionalmente registra un watch.
<code>sync(path)</code>	Fuerza que el servidor conectado alcance las actualizaciones pendientes antes de una lectura posterior.

Las versiones permiten actualizaciones condicionales. Matemáticamente, una escritura condicional puede verse como:

$$\text{setData}(p, d, v) = \begin{cases} \text{éxito,} & \text{si } \text{version}(p) = v, \\ \text{error,} & \text{si } \text{version}(p) \neq v. \end{cases}$$

Esto implementa una forma de *compare-and-set* sobre metadatos distribuidos.

7 Watches: notificaciones sin polling

Un *watch* es un disparador de una sola vez asociado a una sesión. Si un cliente lee un znode con el flag `watch=true`, el servidor promete notificarle cuando cambie la información leída. El watch no transporta el nuevo valor; solo informa que el dato quedó potencialmente obsoleto. Luego, el cliente debe volver a leer y registrar un nuevo watch si desea seguir observando cambios.

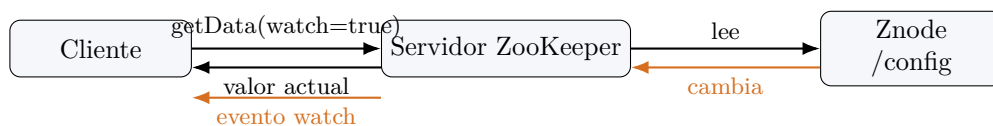


Figure 3: Mecanismo de watches. El cliente recibe un evento y luego decide volver a leer el znode.

8 Garantías de consistencia y orden

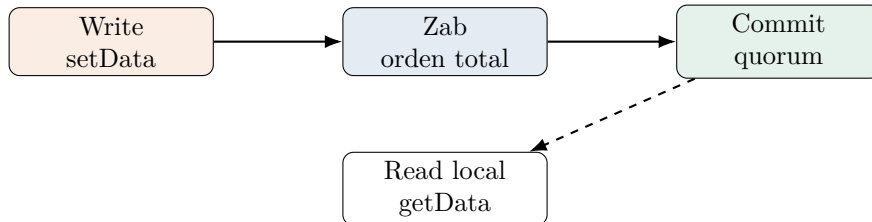
ZooKeeper no ofrece linealizabilidad total para todas las lecturas. Su rendimiento se basa en una decisión arquitectónica: las escrituras se ordenan globalmente, pero las lecturas pueden servirse localmente desde cualquier réplica. Las dos garantías básicas son:

- **Escrituras linealizables:** toda operación que modifica el estado se serializa respetando precedencia.
- **Orden FIFO por cliente:** las operaciones emitidas por un mismo cliente se ejecutan en el orden en que fueron enviadas.

En términos de historia de operaciones, si un cliente emite

$$op_1 \rightarrow op_2 \rightarrow \dots \rightarrow op_k,$$

ZooKeeper garantiza que el servidor observa ese orden para ese cliente. Para escrituras globales, existe un orden total compatible con la precedencia. Para lecturas rápidas, una réplica local puede estar ligeramente retrasada; cuando se necesita una lectura actualizada, se usa `sync()` seguido de `getData()`.



lectura rápida; usar `sync` si se requiere precedencia fuerte

Figure 4: Separación entre escrituras ordenadas por Zab y lecturas locales.

9 Implementación del servicio

El servicio está formado por un *ensemble* de servidores. Cada servidor mantiene una copia en memoria del árbol de znodes. Las escrituras pasan por el líder y se difunden mediante Zab; las lecturas se responden localmente desde la réplica conectada al cliente.

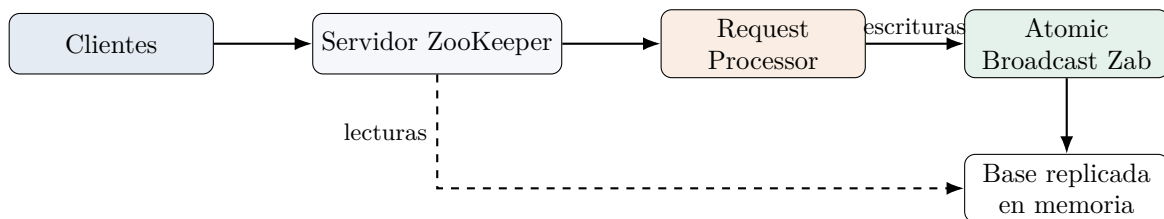


Figure 5: Componentes de alto nivel de ZooKeeper. Las escrituras requieren acuerdo; las lecturas se resuelven localmente.

9.1 Base de datos replicada

La base de datos es completamente replicada y reside en memoria para lograr baja latencia. Para recuperación, el sistema usa un log de escritura anticipada y snapshots periódicos. El paper introduce la idea de *fuzzy snapshots*: no se bloquea todo el árbol para tomar el snapshot; se escanea el árbol y luego se reapiplan transacciones idempotentes en orden durante la recuperación.

9.2 Zab y quorum

Zab es el protocolo de difusión atómica usado para ordenar las actualizaciones. Con $2f + 1$ servidores, el sistema tolera hasta f fallos por caída, siempre que exista una mayoría comunicada. Por ejemplo:

$$n = 2f + 1 \implies f = \left\lfloor \frac{n - 1}{2} \right\rfloor.$$

Así, con 3 servidores se tolera 1 fallo; con 5, se toleran 2 fallos; con 7, se toleran 3 fallos.

10 Recetas de coordinación implementables con ZooKeeper

10.1 Configuración dinámica

Los procesos leen un znode de configuración con watch. Si la configuración cambia, reciben una notificación y vuelven a leer. Este patrón evita hacer polling constante y permite propagar cambios a muchos procesos.

10.2 Membresía de grupo

Cada proceso crea un znode efímero bajo una ruta común, por ejemplo `/grupo/p_i`. Mientras su sesión esté viva, el znode existe. Si el proceso falla, el znode desaparece automáticamente. Otros procesos observan `getChildren(/grupo,true)` para detectar altas y bajas.

10.3 Lock sin efecto manada

La receta más importante del paper es el lock con nodos efímeros secuenciales. Cada cliente crea un znode `lock-000000k`. El cliente con menor número adquiere el lock; los demás solo observan al predecesor inmediato. Así, al liberarse un lock, se despierta como máximo un cliente, evitando el *herd effect*.

Algorithm 1 Lock sin efecto manada usando znodes secuenciales

```

1:  $n \leftarrow \text{create}(l + \text{/lock-}, \text{EPHEMERAL|SEQUENTIAL})$ 
2:  $C \leftarrow \text{getChildren}(l, \text{false})$ 
3: if  $n$  es el znode con menor secuencia en  $C$  then
4:   adquirir lock
5: else
6:    $p \leftarrow$  znode inmediatamente anterior a  $n$ 
7:   registrar exists(p, true) y esperar evento
8:   volver a revisar desde la línea 2
9: end if
10: Para liberar: delete(n)

```



Figure 6: Orden de espera en un lock sin efecto manada. Cada cliente observa solo a su predecesor.

10.4 Barreras dobles

Una barrera doble sincroniza el inicio y el fin de una computación. Al entrar, cada proceso crea un znode hijo; cuando el número de hijos alcanza el umbral, todos pueden comenzar. Al salir, cada proceso elimina su znode; cuando no quedan hijos, la barrera termina.

11 Aplicaciones descritas en el paper

Table 4: Casos de uso presentados por los autores.

Aplicación	Uso de ZooKeeper
Fetching Service de Yahoo!	Coordinación de procesos maestros y fetchers; configuración, salud de nodos y elección de líder.
Katta	Indexador distribuido; seguimiento de esclavos, failover del maestro, asignación de shards y configuración.
Yahoo! Message Broker	Sistema publish-subscribe; distribución de tópicos, membresía, detección de fallos, primary-backup y control operativo.

12 Evaluación experimental

La evaluación se realizó en un clúster de 50 servidores. Los autores midieron throughput, comportamiento ante fallos, latencia y rendimiento de barreras. La característica dominante es que ZooKeeper escala muy bien cuando la carga contiene muchas lecturas, porque las lecturas no pasan por difusión atómica.

12.1 Throughput bajo cargas de lectura

Table 5: Throughput extremo reportado para sistema saturado.

Servidores	100% lecturas	0% lecturas
13	460k ops/s	8k ops/s
9	296k ops/s	12k ops/s
7	257k ops/s	14k ops/s
5	165k ops/s	18k ops/s
3	87k ops/s	21k ops/s

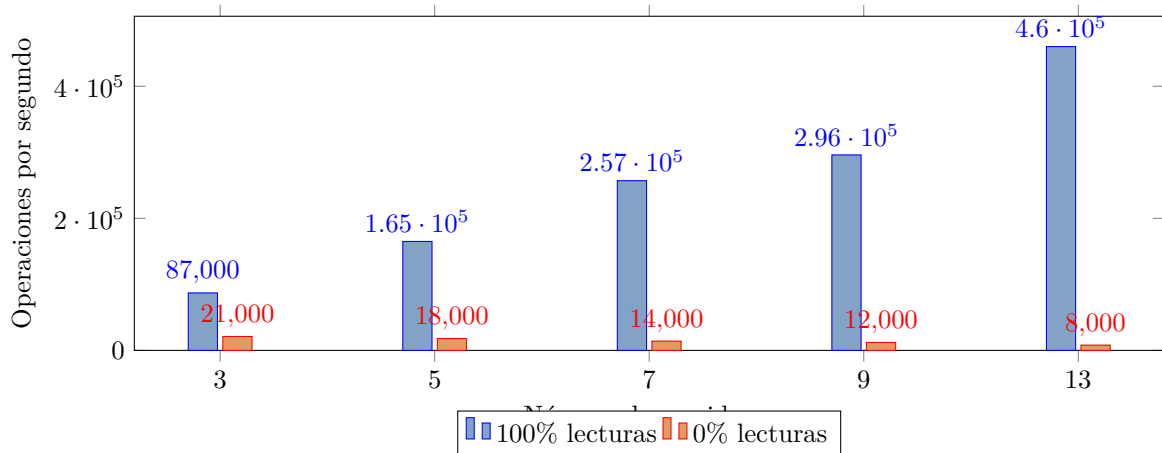


Figure 7: Comparación de throughput extremo. Las lecturas escalan al agregar servidores; las escrituras tienden a degradarse por el costo del acuerdo.

12.2 Latencia y barreras

En pruebas de creación de nodos, el promedio de latencia fue aproximadamente 1.2 ms con tres servidores y 1.4 ms con nueve servidores. En barreras, el tiempo crece casi linealmente con el número de barreras y con el número de clientes, lo cual indica que las recetas construidas sobre ZooKeeper mantienen comportamiento predecible.

13 Comparación conceptual con Chubby

Chubby es un servicio de locks distribuido con garantías fuertes. ZooKeeper comparte la idea de una interfaz tipo sistema de archivos y el uso de replicación, pero toma una decisión diferente: no es un servicio de locks; es un núcleo de coordinación. Esta diferencia tiene consecuencias importantes.

Table 6: Comparación conceptual entre Chubby y ZooKeeper.

Criterio	Chubby	ZooKeeper
Primitiva central	Lock distribuido	Znodes + watches + orden
Consistencia	Más fuerte para el modelo de lock	Escrituras linealizables; lecturas locales rápidas
Acceso de clientes	Tiende a centralizar en el líder	Clientes pueden conectarse a cualquier servidor
Cache/notificación	Invalida caches con leases	Watches de una sola vez, sin bloquear escrituras
Objetivo	Sincronización fuerte mediante locks	Coordinación flexible y alto throughput

14 Análisis crítico

14.1 Fortalezas

- **Simplicidad de API:** pocas operaciones, todas basadas en rutas.
- **Alta escalabilidad en lecturas:** las lecturas locales evitan consenso en el camino crítico.
- **Separación entre núcleo y recetas:** permite construir primitivas nuevas sin modificar el servidor.
- **Buen modelo de fallos por sesión:** los znodes efímeros facilitan membresía y detección de procesos caídos.
- **Orden suficiente:** FIFO por cliente y escrituras linealizables bastan para muchas recetas.

14.2 Limitaciones

- **Lecturas potencialmente obsoletas:** una lectura local puede no ver la última escritura confirmada si no se usa `sync()`.
- **No está diseñado para datos grandes:** los znodes son para metadatos de coordinación, no para archivos o bases de datos generales.
- **Dependencia de mayoría:** si no hay quorum, el servicio no puede garantizar disponibilidad para escrituras.

- **Complejidad trasladada al cliente:** las recetas deben implementarse correctamente para evitar errores de sincronización.

15 Conclusiones

El paper de ZooKeeper muestra una solución elegante para coordinación distribuida: un servicio replicado, pequeño y rápido, con garantías cuidadosamente elegidas. La decisión de no implementar locks como primitiva central evita bloqueos innecesarios y permite que los clientes construyan primitivas adaptadas a su aplicación.

La principal lección de diseño es que un sistema distribuido no siempre necesita garantías fuertes en todas las operaciones. ZooKeeper obtiene rendimiento alto al hacer lecturas locales y reservar el protocolo de acuerdo para las escrituras. Aun así, conserva suficiente estructura de orden para implementar coordinación confiable.

En síntesis, ZooKeeper es un ejemplo representativo de ingeniería de sistemas distribuidos: combina replicación, consenso práctico, sesiones, notificaciones y una API mínima para resolver un conjunto amplio de problemas reales de coordinación.

References

- [1] Patrick Hunt, Mahadev Konar, Flavio P. Junqueira y Benjamin Reed. *ZooKeeper: Wait-free coordination for Internet-scale systems*. Yahoo! Grid y Yahoo! Research.